

Project #4. File System Enablement on OpenSSD via Linux Kernel Modifications and Evaluation of File-Based KVSs and KV-SSDs

OBJECTIVE

In Project #3, you implemented a Key-Value SSD (KV-SSD) on the Cosmos+ OpenSSD platform. Building on this work, the objective of this project is to analyze how different storage abstractions affect the I/O path and performance by comparing a KV-SSD with a conventional file system-based Key-Value Store (KVS) running on the host. To enable this comparison, one of standard Linux file systems, `ext4` [1], must first operate correctly on the OpenSSD device. However, creating an `ext4` file system fails under the unmodified Linux kernel of your (typically `v5.15.xxx`). One of the main goals of this project is to identify the root cause of this failure and fix it through host-side kernel modifications. In this project, the issue must be resolved at **the Linux kernel level**, *not by modifying the device firmware*.

To achieve this, students are expected to analyze the OpenSSD firmware behavior together with the `ext4` Linux file system [4], block layer [3], and NVMe driver [2]. While the required source code changes are relatively *small*, identifying the root cause requires careful and thorough analysis. *Premature or blind modifications are discouraged*. After enabling successful file system creation, students will evaluate the performance of the provided file-based KVS and compare it with the KV-SSD from Project #3. Through this process, the project aims to reveal the performance characteristics and trade-offs imposed by different storage abstractions.

Duration. June 11 (Thu) – June 26 (Fri) (2 Weeks)

TASK DESCRIPTION

Task 1: Enabling File System Creation on the Cosmos+ OpenSSD Platform.

In this task, students will enable standard Linux file system support on the Cosmos+ OpenSSD platform by modifying and rebuilding the Linux kernel. By default, the Linux kernel provided on the lab PCs fails to create an `ext4` file system on the OpenSSD device. Your goal is to analyze this failure, identify its root causes, and apply appropriate kernel-level fixes so that `mkfs.ext4` executes successfully.

- Investigate why running `mkfs.ext4` on the Cosmos+ OpenSSD device fails under the unmodified kernel. This typically involves tracing how file system creation interacts with the block layer, NVMe driver, and request handling paths for the OpenSSD device.
- Identify and modify the relevant Linux kernel source files to resolve the observed issues. You may inspect and change any kernel subsystem that is necessary, including (but not limited to) the NVMe driver or block layer logic.
- Before making any code changes, **carefully read** `PATCH_GUIDE.md` and follow the prescribed Git-based workflow. You must initialize a Git repository immediately after extracting the kernel source and manage all changes as commits that can later be exported as a patch.
- Build and install a **new custom kernel** rather than modifying the existing one. The original kernel must remain intact so that the system can always be recovered by selecting it from GRUB.
- Boot into the newly installed kernel and verify that the kernel version reflects your custom build (e.g., `5.15.190-openssd`). Then, confirm that `mkfs.ext4` completes successfully on the Cosmos+ OpenSSD device under this kernel.

Checklist.

- All kernel modifications must be applied to Linux kernel version **5.15.190**. Using a different kernel version is **not allowed**.
- You must not modify or remove the original lab kernel. A separate kernel build with a custom local version suffix (e.g., `-openssd`) is required.

- Your submission must include:
 - A clean Git history containing only your relevant kernel modifications.
 - A patch file generated according to the instructions in `PATCH_GUIDE.md`.
- After booting into your custom kernel:
 - `uname -r` must correctly report the modified kernel version.
 - Running `mkfs.ext4` on the Cosmos+ OpenSSD device must complete without errors.
- Kernel crashes, device aborts, or accidental modification of the original lab kernel are **penalized**.

Task 2: Comparative Evaluation of File-Based KVS and KV-SSDs.

After the successful creation of file systems over the OpenSSD, you will conduct a comparative performance evaluation between (1) a file-based Key-Value Store (KVS) implemented on top of a file system and (2) a Key-Value SSD (KV-SSD) accessed through native KV commands. The goal of this task is to understand the performance implications of the traditional file system I/O stack versus KV-native storage access.

- Evaluate the performance of a file-based KVS using the provided `kv_fs_bench` program. This benchmark operates on top of a standard Linux file system, `ext4`, and represents a typical host-managed KVS deployment.
- Perform experiments under multiple synchronization configurations by modifying the `KV_FSYNC_PERIOD` parameter defined in `kv_fs_backend.h`. Specifically, you must evaluate all of the following cases:
 - `KV_FSYNC_PERIOD = 1`
 - `KV_FSYNC_PERIOD = 10`
 - `KV_FSYNC_PERIOD = 100`
 - `KV_FSYNC_PERIOD = 1000`
- For each `KV_FSYNC_PERIOD` setting, run `kv_fs_bench` with the following parameters:

(number of operations, keyspaces) = (100000, 524288)

and collect the full benchmark output. Screenshots or terminal captures of all runs are required.

- Then, evaluate the performance of a KV-SSD using the `kv_bench` program from Project #3. This benchmark issues native NVMe KV commands directly to the KV-SSD, bypassing the file system and block I/O stack.
- Run `kv_bench` using the same parameters:

(100000, 524288),

and include the execution results as part of your report. Students must use their Project #3 implementation as the KV-SSD firmware. If the original implementation does not fully support this parameter configuration, additional fixes or extensions must be implemented during Project #4.

Checklist.

- All four `KV_FSYNC_PERIOD` configurations (1, 10, 100, 1000) must be evaluated for the file-based KVS. Missing any configuration will result in a penalty.
- Benchmark results must include clear evidence of execution, such as terminal output captures, and must correspond exactly to the specified parameters (100000, 524288).
- If your Project #3 KV-SSD firmware is incomplete or fails to correctly execute `kv_bench` under the required parameter setting, you must update the firmware and submit it as a separate archive (e.g., `prj4-ssd-20241234.tar.gz`) together with the Project #4 submission.
- Your report must include a comparative analysis of the evaluation results:
 - Discuss how performance changes as `KV_FSYNC_PERIOD` increases in the file-based KVS.
 - Compare these results against the KV-SSD evaluation results.
 - Explain the observed performance differences in terms of I/O path length, synchronization cost, file system overhead, and KV-native access semantics.
- Superficial comparisons or result listings without analysis are **penalized**.

MATERIALS PROVIDED

- **Host-side evaluation program** (`prj4-host-evaluation/`): Includes `Makefile`, `kv_fs_bench.cc`, `kv_fs_backend.(h|cc)`, and `README.md`. This binary issues random KV I/O workloads just like the one provided in the previous phase.
- **Linux Kernel Development Tips** (`prj4-host-kernel-dev/`): Includes `README.md` and `PATCH_GUIDE.md`. You **MUST** read these markdown files *carefully* to follow the setup and environmental rules defined in this project phase.

STUDENT SUBMISSION

Submit the following items under the naming convention below:

- 1) **A patch file that contains the differences between the original kernel source, and your modified one** (see the [PATCH_GUIDE.md](#) carefully) (e.g., `prj4-kernel-{studentID}.patch`)
- 2) **Project Report (exported to PDF)** (e.g., `prj4-document-{studentID}.pdf`)

Submission Window. Opens: June 11 (Thu) 15:00 Closes: June 26 (Fri) 23:59

EVALUATION CRITERIA

- **Functionality (60%)** — `ext4` file system must be created successfully on the Cosmos+ OpenSSD platform after reflecting your kernel source modifications written in the submitted patch file. After, the benchmark must complete without crashes or hangs, and the final result must report OK for all operations with no FAIL entries.
- **Report Quality (40%)** — clarity, root cause of file system creation fails, explanation of your kernel modifications, and performance comparison between a file-based KVS and a KV-SSD.

NOTES

- Use administrative privileges (`sudo`) only when absolutely necessary, and always double-check before executing commands.
- Handle the Cosmos+ OpenSSD board with care. Physical mishandling can permanently damage the hardware.
- Any destructive damage to the board or the Linux development machine — whether physical or software-induced — is entirely your responsibility. Such incidents may delay your progress, but all delays and losses during project execution must be borne by you alone.

REFERENCES

- [1] Mingming Cao, Suparna Bhattacharya, and Ted Ts'o. Ext4: The next generation of ext2/3 filesystem. In *2007 Linux Storage & Filesystem Workshop (LSF 07)*, San Jose, CA, February 2007. USENIX Association.
- [2] The Linux Kernel Community. Nvme host driver implementation. <https://github.com/torvalds/linux/blob/master/drivers/nvme/host/>. accessed: 2025-12-09.
- [3] The Linux Kernel Community. Block multi-queue (blk-mq) subsystem. <https://github.com/torvalds/linux/tree/master/block>. accessed: 2025-12-09.
- [4] The Linux Kernel Community. ext4 file system. <https://github.com/torvalds/linux/tree/master/fs/ext4>. accessed: 2025-12-09.